SEMANTIC SEARCH STUDIO

CASE STUDY: TRADITIONAL KEYWORD SEARCH VS AI-POWERED SEMANTIC SEARCH USING CHROMADB

A PROJECT REPORT

Submitted by

ISHAAN SAHU [RA2311003010352]

PRATEEK SINHA [RA2311003010318]

ARNAV AMRIT [RA2311003010299]

APEKSHA SACHAN [RA2311003010306]

SAVIO BINO [RA2311003010291]

Under the Guidance of

[Project Guide Name]

[Designation]

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

COMPUTER SCIENCE AND ENGINEERING

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

SCHOOL OF COMPUTING

DEPARTMENT OF COMPUTING TECHNOLOGIES

COLLEGE OF ENGINEERING AND TECHNOLOGY

KATTANKULATHUR - 603 203

NOVEMBER 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR - 603 203

BONAFIDE CERTIFICATE

Certified that the project report titled "Case Study: Traditional Keyword Search vs AI-Powered Semantic Search Using ChromaDB" is the bona fide work of ISHAAN SAHU [RA2311003010352], PRATEEK SINHA [RA2311003010318], ARNAV AMRIT [RA2311003010299], APEKSHA SACHAN [RA2311003010306], SAVIO BINO [RA2311003010291], who carried out the project work under the guidance of [Project Guide Name and Designation] in partial fulfillment of the requirements for the Bachelor of Technology degree in Computer Science and Engineering.

The project implements Semantic Search Studio, a local document-search application that compares traditional BM25 keyword ranking with embedding-based semantic retrieval using ChromaDB. The signatures below are to be completed by the project guide and the department before formal submission.

SIGNATURE
[Project Guide]

Name and designation
Department of Computing Technologies

SIGNATURE
[Head of Department]

Name and designation
Department of Computing Technologies

ABSTRACT

This project develops Semantic Search Studio, a local document-search application and a case study comparing BM25 keyword retrieval with embedding-based semantic retrieval over the same text passages. The application accepts PDF, DOCX, and TXT files, extracts text, creates sentence-aware word-count chunks, stores metadata and evaluation records in SQLite, and stores normalized sentence embeddings in persistent ChromaDB. Users can search by exact terms or meaning, compare ranked results, record graded relevance judgments, calculate ranking metrics, and inspect query timings.

The system is organized as a Streamlit interface, service orchestration, document-processing and retrieval modules, SQLite persistence, and a local Sentence Transformers model connected to ChromaDB. Its evaluation workflow is designed to keep corpus, chunks, queries, and top-K constant between methods. The report documents architecture, implementation, test evidence, and a paired evaluation protocol.

The current automated software suite completed 13 tests with no failures. The local study database contains no indexed evaluation corpus, relevance judgments, or completed retrieval experiment. Therefore no comparative Precision@K, Recall@K, MRR, NDCG@K, or latency result is claimed. Those results require corpus preparation, complete judgments, and a paired run.

Keywords: Information retrieval, BM25, semantic search, Sentence Transformers, ChromaDB, evaluation

TABLE OF CONTENTS

ABSTRACT iii
LIST OF FIGURES v
LIST OF TABLES v
ABBREVIATIONS vi
CHAPTER 1 INTRODUCTION 1
CHAPTER 2 LITERATURE SURVEY 3
CHAPTER 3 METHODOLOGY 5
CHAPTER 4 RESULTS AND DISCUSSIONS 9
CHAPTER 5 CONCLUSION AND FUTURE ENHANCEMENTS 12
REFERENCES 13
APPENDIX 14
SCREENSHOTS OF MODULES 17

LIST OF FIGURES

Figure No.	Title of the Figure	Page
1.1	Two retrieval paths and a paired comparison	1
3.1	Semantic Search Studio system architecture	5
3.2	Document-to-evaluation workflow	5

LIST OF TABLES

Table No.	Title of the Table	Page
3.1	Functional requirements and implementation status	6
3.2	Persistent data entities	6
3.3	Evaluation metrics and interpretation	7
4.1	Automated functional verification summary	10
4.2	Current implementation and study status	10
A.1	Environment configuration	14
B.1	Starter query examples	14
C.1	Source module map	15
C.2	Automated test modules	15

ABBREVIATIONS

Abbreviation	Expansion
BM25	Best Matching 25 (probabilistic ranking function)
CPU	Central Processing Unit
CSV	Comma-Separated Values
DOCX	Microsoft Word Open XML document format
IDF	Inverse Document Frequency
IR	Information Retrieval
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
OCR	Optical Character Recognition
PDF	Portable Document Format
SQLite	Embedded relational database engine
UI	User Interface

CHAPTER 1
INTRODUCTION

1.1 Introduction

Information retrieval systems rank documents or passages in response to a query. Traditional lexical systems such as BM25 rely on the occurrence and distribution of query terms. They are often effective when a query contains the exact names, identifiers, or technical vocabulary used in a document. Dense semantic retrieval represents queries and passages as vectors and can retrieve related text even when the wording differs.

Semantic Search Studio is a local workspace for examining these complementary signals on a user-provided document collection. It offers keyword search, semantic search, side-by-side comparison, and a human-judgment workflow. This makes the project both a software deliverable and a practical case study: the interface supports comparison, while the report records the conditions required for a fair evaluation.

Lexical retrieval	Semantic retrieval	Paired case study
BM25 scores term overlap and term rarity.	Sentence embeddings represent contextual similarity.	Same passages, query, and top-K are compared.

Fig 1.1: Two retrieval paths and a paired comparison

1.2 Problem Statement

Users searching local documents may miss useful passages when they rely only on exact terms, while semantic systems may overlook rare names, codes, or exact phrases. A useful comparison must expose both methods on the same corpus, passage boundaries, query set, and result cutoff. It also needs a transparent judgment method and clear latency definitions; without those controls, results can be difficult to interpret or reproduce.

1.3 Objective

The objective is to build and document a local application that supports controlled comparison between BM25 keyword retrieval and embedding-based semantic search using ChromaDB. The project goals are:

1. Accept PDF, DOCX, and TXT documents and create searchable passages with source metadata.
2. Provide BM25 retrieval and local embedding retrieval over a shared passage collection.

3. Present method-specific results and paired rankings in a readable browser interface.
4. Support query categorization, 0–3 relevance grades, ranking metrics, latency measurements, and CSV export.
5. Persist documents, search history, judgments, experiment configuration, and results locally.
6. Report observed software verification accurately and distinguish it from a retrieval-quality study.

1.4 Tools and Technologies Used

The application combines the following technologies and libraries:

- Python for the application, retrieval logic, document processing, metrics, and experiment orchestration.
- Streamlit for the guided browser workspace, forms, charts, and result views.
- Sentence Transformers with all-MiniLM-L6-v2 for local 384-dimensional sentence embeddings.
- ChromaDB for persistent vector storage and cosine-distance nearest-neighbour retrieval.
- SQLite for document and chunk metadata, search history, queries, relevance judgments, and evaluations.
- PyMuPDF, python-docx, and UTF-8 handling for PDF, DOCX, and TXT extraction.
- Plotly and pandas for evaluation charts and tables; pytest and Streamlit AppTest for automated verification.

1.5 Scope and Boundaries

The implemented scope includes local ingestion, extraction, sentence-aware word-count chunking, BM25 ranking, local embedding and Chroma retrieval, paired search, query history, graded judgments, evaluation metrics, timing, and CSV export. Optical character recognition for scanned PDFs, hosted language-model answers, hybrid ranking, and a multi-user production deployment are outside the current build. The separate RDFLib triple example is educational and does not participate in the retrieval pipeline.

CHAPTER 2
LITERATURE SURVEY

2.1 Traditional Keyword Retrieval and BM25

BM25 is a probabilistic term-weighting method that scores a passage using query-term matches, term frequency, inverse document frequency, and passage-length normalization [1]. Its saturation and length parameters keep repeated terms and long passages from dominating without bound. In this project the defaults are k1 = 1.5 and b = 0.75. The result is a ranking score, not a probability or confidence value.

The implementation lowercases Unicode text and extracts word-like tokens. It builds document frequency and average passage length over the current chunk set, then scores matching query terms. This transparent baseline is especially relevant for exact phrases, identifiers, and domain vocabulary. Language-specific stemming and synonym expansion are outside the current scope.

2.2 Sentence Embeddings and Semantic Retrieval

Sentence-BERT uses a siamese or triplet network design to produce sentence embeddings that can be compared with cosine similarity [2]. The project uses the all-MiniLM-L6-v2 model through Sentence Transformers; its model card describes 384-dimensional embeddings and semantic-search applications [3]. Similarity is derived from a learned representation, so passages can match a query without sharing every word.

Embedding distance is not a calibrated measure of relevance. A model can overlook exact identifiers, negation, uncommon abbreviations, or domain distinctions. For this reason, the project displays semantic and lexical rankings separately and evaluates them using explicit human judgments rather than asserting that one method is universally superior.

2.3 Vector Databases and ChromaDB

A vector collection associates each passage with an identifier, text, metadata, and vector. Chroma collections support nearest-neighbour queries and persistence between application restarts [4]. Semantic Search Studio stores normalized vectors and source metadata in a persistent cosine-distance collection; lower cosine distance ranks closer vectors first. The current configuration uses Chroma's legacy `hnsw:space` metadata form, which should be revisited during a dependency migration [5].

2.4 Evaluation of Ranked Retrieval

Information-retrieval evaluation compares a ranked list with relevance judgments. Precision@K measures the fraction of the top K results marked relevant, Recall@K measures the fraction of known relevant passages retrieved, MRR emphasizes the rank of the first relevant passage, and NDCG@K accounts for graded relevance and rank discounting [6]. These metrics answer different questions and should be read together with the corpus, cutoff, query categories, judgment coverage, and timing scope.

2.5 Position of This Project

The project is an applied comparison workspace rather than a new ranking algorithm. Its contribution is the integration of lexical search, vector search, aligned side-by-side comparison, manual graded judgments, experiment records, and local exports. The evaluation gap is empirical: until the team supplies a representative corpus, categorized queries, and complete judgments, the implementation cannot support a conclusion about which method ranks better or runs faster.

CHAPTER 3
METHODOLOGY

3.1 Case-Study Method

The planned study uses a paired design. Both retrieval methods operate on the same indexed passages and receive the same query and top-K cutoff. Queries should cover paraphrases, exact terms, identifiers, and domain terminology. Assessors grade each corpus chunk from 0 (not relevant) to 3 (highly relevant) and record coverage. The report must identify the data source, software configuration, hardware, and measurement scope used in each experiment.

3.2 System Architecture

The application uses a layered local architecture. Streamlit presents the workspace; the service layer coordinates queries and timing; core modules process documents and run BM25 or semantic retrieval; SQLite stores relational records; and ChromaDB persists vectors and passage metadata.

Layer	Components and responsibility
Input	PDF, DOCX, and TXT uploads; query text and top-K selection.
Processing	Text extraction, normalization, sentence-aware chunking, source/page metadata.
Application	Streamlit UI, search service, keyword and semantic engines.
Persistence	SQLite for documents, chunks, history, judgments, experiments; ChromaDB for vectors and metadata.
Evaluation	Graded judgments, Precision@K, Recall@K, MRR, NDCG@K, repeated timings, CSV export.

Fig 3.1: Semantic Search Studio system architecture

Stage 1	Stage 2	Stage 3	Stage 4
Add and index documents	Run keyword or semantic query	Compare both result lists	Judge relevance and evaluate

Fig 3.2: Document-to-evaluation workflow

3.3 Functional Requirements

Requirement	Implementation evidence	Status
Ingest documents	Upload and extract PDF, DOCX, TXT; configurable size limit.	Implemented
Build a shared passage set	Sentence-aware chunking with source and optional page metadata.	Implemented

Requirement	Implementation evidence	Status
Search using both methods	BM25 and Chroma cosine retrieval over stored chunks.	Implemented
Compare ranked outputs	Same query and K shown side-by-side.	Implemented
Evaluate relevance	Query categories, 0–3 grades, metrics, timing, coverage.	Workflow implemented ; study data pending
Export experiment	Selected evaluation results can be exported to CSV.	Implemented ; no completed experiment

Table 3.1: Functional requirements and implementation status

3.4 Data Design and Persistence

SQLite stores document identity, extracted chunks, search history, test queries, relevance judgments, experiment configuration, and per-query evaluation results. ChromaDB separately stores passage vectors, text, and source metadata. Deleting a document removes its vector entries and cascades its relational chunk and judgment records. Because SQLite and ChromaDB do not share an atomic transaction, an abrupt interruption during indexing can require cleanup and re-indexing.

Entity	Purpose
documents	File name, type, hash, timestamp, indexing status, page/chunk totals, errors.
chunks	Document-scoped passage ID, order, source page, extracted text.
search_history	Query, method, result count, saved results, and timing.
test_queries and relevance_judgments	Query category and explicit per-chunk 0–3 relevance grades.
experiments and evaluation_results	Configuration, per-query metrics, timing variation, and judgment coverage.

Table 3.2: Persistent data entities

3.5 Document Processing and Retrieval

PDF extraction uses PyMuPDF and retains one-based page references; DOCX processing reads paragraphs and table cells; TXT processing uses UTF-8 with malformed bytes replaced. Scanned PDFs without extractable text produce an explicit OCR limitation. The chunker

normalizes whitespace and builds sentence-aware passages using a default 500-word window with 75-word overlap; these are word counts rather than model-token limits.

The embedding engine lazily loads all-MiniLM-L6-v2, encodes passages in batches, and normalizes vectors. ChromaDB returns cosine distances, where lower distance ranks closer vectors first. The BM25 engine lowercases Unicode word tokens and uses k1 = 1.5 and b = 0.75. Its in-memory index is rebuilt for a keyword query, a cost included in the end-to-end timing scope.

3.6 Query Design and Relevance Protocol

1. Choose a legally usable corpus representative of the intended search domain; record its source, file types, document and chunk counts, language, and exclusions.
2. Prepare balanced queries for paraphrases, exact wording, identifiers, and domain terminology. The supplied starter file has three prompts and is not the final study set.
3. Freeze the corpus, chunk size, and overlap before grading; recheck judgments if those settings or documents change.
4. Grade every chunk from 0 through 3 and leave Unjudged only when an assessor has not reviewed it. Report judgment coverage and assessor procedure.
5. Run BM25 and semantic methods on each query with the same K and corpus. Save configuration and export the per-query results.

3.7 Metrics and Timing Scope

Measure	Interpretation	Caution
Precision@K	Relevant share among the top K retrieved passages.	Denominator is configured K, even if fewer results are returned.
Recall@K	Known relevant passages retrieved within K.	Undefined when no relevant passage is judged.
MRR	Reciprocal rank of the first relevant result.	Zero if none appears in top K.
NDCG@K	Graded relevance with rank discounting.	Uses gain 2^grade - 1.
Latency	Repeated timings summarized by mean and standard deviation.	Name retrieval-only or end-to-end scope.

Table 3.3: Evaluation metrics and interpretation

Retrieval-only latency measures BM25 scoring or the Chroma query. End-to-end query latency includes BM25 in-memory index construction plus scoring, or query embedding plus Chroma

retrieval. Model loading, shared SQLite chunk loading, document extraction, and indexing are excluded from both. The default evaluation repeats timings three times. Unjudged chunks are treated as grade zero by the metric implementation, so incomplete judgments can bias results.

3.8 Tools and Runtime Configuration

The application targets Python 3.10 or 3.11 and runs locally on CPU, with optional CUDA. The default embedding model is all-MiniLM-L6-v2, default top K is 5, and upload limit is 50 MB per file. The model loads on its first semantic request and may require a network download on first use. See Appendix A for setup and environment overrides.

CHAPTER 4
RESULTS AND DISCUSSIONS

4.1 Features Demonstrated

The repository implements the main application workflow: local document ingestion, library management, lexical and semantic search, paired comparison, search history, relevance grading, experiment execution, charts, and CSV export. The interface uses a guided navigation flow so a user can add documents, search, compare methods, and proceed to evaluation.

- Upload and index PDF, DOCX, and TXT files; preserve source names and available PDF pages.
- Inspect document records and remove indexed documents with their related stored records.
- Search with BM25 or semantic embeddings and review ranked passages, sources, and scores.
- Compare both methods on a common query and top-K value.
- Create categorized test queries, enter graded judgments, run experiments, inspect metrics, and export results.

4.2 Functional Test Results

The automated suite was run on 5 November 2026 using Python 3.10.0 and pytest 8.4.2. It completed in 9.36 seconds with 13 passed and 0 failed. Tests exercise controlled fixtures and an AppTest UI flow. The vector-store test uses deterministic test vectors; it does not load the production Sentence Transformer model or measure production retrieval effectiveness.

Test area	Cases	Observed outcome
Application UI	1	Overview, upload/library, keyword search, and gated compare navigation rendered without exception.
Chunking	2	Boundaries, overlap, metadata, and edge cases passed.
Persistence	1	Document/chunk records, history, evaluation persistence, and deletion cascade passed.
Document processing	4	TXT, DOCX/PDF extraction and expected empty, unsupported, corrupt/scanned input paths passed.
Evaluation metrics	2	Binary and graded metric calculations and empty-judgment behavior passed.
Keyword retrieval	2	Tokenization and BM25 ranking passed.

Test area	Cases	Observed outcome
Vector retrieval	1	Deterministic-vector Chroma insert, query, and delete path passed.
Total	13	13 passed; 0 failed; 9.36 seconds.

Table 4.1: Automated functional verification summary

4.3 Current Evaluation Status

Area	Observed status	Evidence or next step
Application interface	Implemented and launched locally for review.	Streamlit app with guided navigation and workspace pages.
Supported file types	Implemented.	PDF, DOCX, TXT; scanned PDF OCR is unavailable.
Paired search workflow	Implemented.	Both methods receive the same query and top K.
Indexed study corpus	None in the local database.	Select and index a representative collection.
Relevance judgments	None recorded.	Grade corpus chunks per query and report coverage.
Retrieval-quality metrics	Not measured.	Run paired experiments after judgments are complete.
Latency results	Not measured.	Measure and report both defined timing scopes.

Table 4.2: Current implementation and study status

At report preparation, the local SQLite database contains zero indexed documents, chunks, searches, test queries, relevance judgments, experiments, or evaluation results. Therefore no measured Precision@K, Recall@K, MRR, NDCG@K, or latency values are available. The current evidence supports software functionality within the tested paths, not a conclusion that one retrieval method is more accurate or efficient.

4.4 Advantages

The local workflow keeps document text and vector data in configured workstation storage. A paired comparison exposes term overlap and semantic similarity on the same passages, while source metadata, persistent history, graded judgments, and experiment exports support repeatable review. Separate retrieval-only and end-to-end timing scopes also make performance statements easier to interpret.

4.5 Limitations and Discussion

The current build is at the application and evaluation-protocol stage. Its retrieval-quality claims remain open until a representative corpus and complete judgments are prepared. A small demonstration set cannot establish domain coverage. Incomplete labels can bias metrics because unjudged passages are treated as grade zero. Scanned PDF OCR is not supported. BM25 rebuilds its in-memory index for each query, which may affect end-to-end latency at larger corpus sizes. SQLite and ChromaDB cannot share one atomic transaction, and an interrupted ingestion can require cleanup and re-indexing. The default model and tokenizer have not been compared with domain-specific alternatives.

The functional tests show that the covered code paths execute correctly with controlled data; they do not measure ranking quality or production scale. BM25 can be a strong fit for exact names and identifiers, while embeddings can surface passages that use related wording. The appropriate method depends on the query and corpus, so conclusions should follow the paired judgment results rather than the method label. End-to-end timings must include query embedding or BM25 index construction as defined in Chapter 3.

CHAPTER 5
CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

Semantic Search Studio implements a local document-search application that compares BM25 keyword ranking with embedding-based cosine retrieval through ChromaDB. It covers document processing, shared chunking, paired search, query history, graded relevance judgments, ranking metrics, timing definitions, and experiment export. Automated functional verification passed 13 tests in the recorded environment.

The case study is not yet empirically complete. No study corpus, judgments, or completed evaluation runs are present in the local project database. The current evidence therefore supports the implementation and tested workflows, but not comparative accuracy, speed, or superiority. The evaluation protocol in Chapter 3 defines the work needed to answer the project's research questions.

5.2 Future Enhancements

- Select a representative, licensed corpus and record its source, composition, and indexing configuration.
- Prepare a balanced query set covering paraphrases, exact wording, identifiers, and domain terminology.
- Complete and review 0–3 judgments for every corpus chunk; record assessor procedure and judgment coverage.
- Run paired experiments and add observed per-query and aggregate quality and latency results to the report.
- Measure indexing and query behavior at several corpus sizes and document the hardware and runtime environment.
- Consider OCR, hybrid lexical/vector ranking, cached BM25 statistics, model comparison, and a Chroma configuration migration.

REFERENCES

[1] S. E. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009. doi: 10.1561/1500000019.

[2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proceedings of EMNLP-IJCNLP 2019, pp. 3982–3992. doi: 10.18653/v1/D19-1410.

[3] Sentence Transformers, “all-MiniLM-L6-v2 model card.” https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 (accessed 5 November 2026).

[4] Chroma, “Collections - Chroma Cookbook.” https://cookbook.chromadb.dev/core/collections/ (accessed 5 November 2026).

[5] Chroma, “Configuration - Chroma Cookbook.” https://cookbook.chromadb.dev/core/configuration/ (accessed 5 November 2026).

[6] U.S. National Institute of Standards and Technology, “trec_eval: TREC evaluation software.” https://github.com/usnistgov/trec_eval (accessed 5 November 2026).

[7] Streamlit, “App model summary.” https://docs.streamlit.io/get-started/fundamentals/summary (accessed 5 November 2026).

[8] CzPhantom10, “semantic-search-case-study,” project source repository. https://github.com/CzPhantom10/semantic-search-case-study (accessed 5 November 2026).

APPENDIX

Appendix A: Project Setup and Configuration

From the repository root, create a virtual environment, install the declared dependencies, and launch the local Streamlit workspace:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
streamlit run app.py
```

Environment variable	Purpose
SEMSEARCH_DATA_DIR	Override the runtime data directory.
SEMSEARCH_MODEL	Select the embedding model; re-index after changing it.
SEMSEARCH_CHUNK_SIZE	Set the word-count passage window.
SEMSEARCH_CHUNK_OVERLAP	Set the word-count overlap.
SEMSEARCH_TOP_K	Set the default number of results.
SEMSEARCH_LATENCY_RUNS	Set repeated evaluation timing count.
SEMSEARCH_MAX_UPLOAD_MB	Set the per-file upload size limit.
SEMSEARCH_DEVICE	Choose CPU/CUDA or leave automatic selection.

Table A.1: Environment configuration

The first semantic request may download the configured model. CPU execution is supported and CUDA is optional. Runtime uploads and databases are stored in the configured local data directory and are excluded from version control.

Appendix B: Starter Query Examples

The supplied CSV contains demonstration prompts, not a final evaluation set. Review categories and confirm that the answers appear in the selected corpus before formal judgments.

Table B.1: Starter query examples

Query	Suggested category	Expected information
How can I protect my computer from hackers?	paraphrase	Network-security techniques that prevent attacks.
What security methods prevent cyberattacks?	lexical	Protective network-security methods.

Query	Suggested category	Expected information
What does multi-factor authentication do?	specific concept	An authentication measure that protects accounts.

Appendix C: Source and Test Inventory

Module	Responsibility
app.py	Streamlit entry point and guided workspace navigation.
ui/	Shared presentation components and visual styling.
core/document_processor.py	PDF, DOCX, and TXT extraction.
core/chunker.py	Text normalization and word-count passage generation.
core/keyword_engine.py	BM25 tokenization and ranking.
core/embedding_engine.py and vector_store.py	Local sentence embeddings and Chroma persistence/query.
core/evaluation.py	Ranking metric calculations.
services/search_service.py	Search orchestration, comparison, and timing.
database/db_manager.py	SQLite persistence and related record management.

Table C.1: Source module map

Test module	Coverage summarized from the suite
test_app.py	Overview, upload/library path, keyword search, gated compare navigation.
test_chunker.py	Chunk boundaries, overlap, metadata, and edge cases.
test_database.py	Document/chunk records, history, evaluation persistence, deletion cascade.
test_document_processor.py	TXT, DOCX/PDF extraction and error paths.
test_evaluation.py	Binary and graded metric behavior, including empty judgments.
test_keyword_search.py	Tokenization and BM25 ranking.
test_semantic_search.py	Deterministic-vector Chroma insert, query, and deletion.

Table C.2: Automated test modules

Appendix D: Glossary of Terms

- BM25: a probabilistic lexical ranking function based on query-term evidence and length normalization.
- Chunk: a passage created from a source document for indexing and retrieval.

- Embedding: a numeric vector representation of text used to compare semantic similarity.
- Judgment: a human-assigned relevance grade for a passage and query pair.
- NDCG: a ranking measure that accounts for graded relevance and position; Top K: the number of highest-ranked passages returned for one query.

SCREENSHOTS OF MODULES

Workspace Overview

The overview introduces the local workspace and guides a user through document ingestion, search, method comparison, and evaluation. Insert a current application screenshot here after the team's walkthrough.

Document Library and Upload

The library lists indexed files and their processing status. The upload view accepts PDF, DOCX, and TXT documents and reports extraction or indexing errors. Insert current screenshots of these views here.

Search and Method Comparison

Keyword and semantic search present ranked passages with source metadata and method-specific scores. The comparison view places both rankings beside one another for a common query. Insert current screenshots of these views here.

Evaluation Workbench

The evaluation workbench supports categorized test queries, passage grading, paired experiments, metrics, timing charts, and CSV export. Insert a current screenshot after an experiment has been prepared with a study corpus.